UCAM

Práctica final

Cálculo Paralelo de gradientes de funciones multivariables con apoyo de inteligencia artificial generativa en el desarrollo

Programación Paralela

Grado en Ingeniería Informática

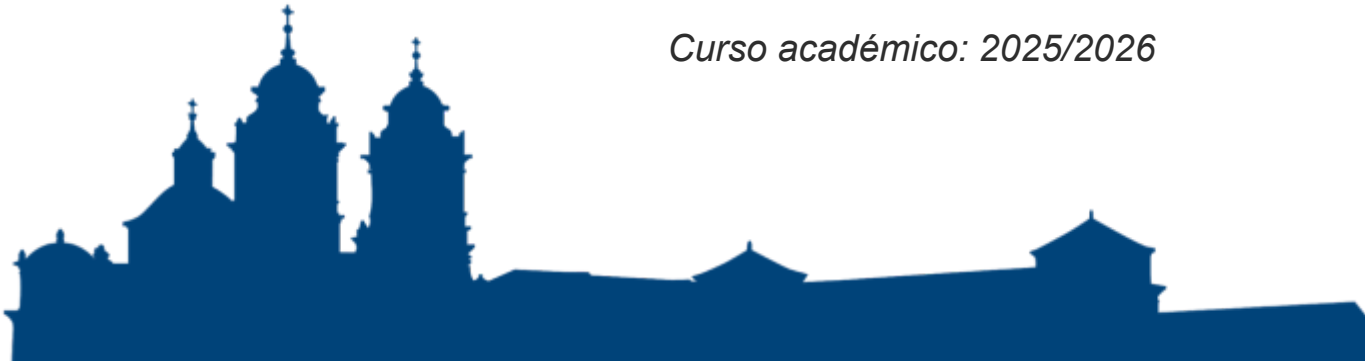
por:

Santiago Colomer Andreu

Vicente Losana Moreno

Docente: Baldomero Imbernón Tudela

Curso académico: 2025/2026



Índice de contenidos

1. Introducción....2
2. Descripción del problema....2
2.1. Definición matemática....2
2.2. Restricciones y supuestos del problema....2
3. Diseño e implementación del algoritmo....3
3.1. Estructura general del programa....3
3.2. Implementación secuencial....4
3.3. Paralelización en CPU con OpenMP....5
3.3.1. Paralelización principal....6
3.3.2. Ventajas y limitaciones....6
3.4. Paralelización en GPU con CUDA....7
4. Resultados Experimentales....9
4.1. Entorno de ejecución....9
4.2. Resultados....10
4.3. Análisis de los resultados....10
5. Conclusiones....12
6. Bibliografía....12

1. Introducción

El objetivo de esta práctica es implementar y analizar el cálculo del gradiente de funciones multivariables, comparando una versión secuencial con versiones paralelizadas mediante OpenMP y CUDA. Además, la solución se extiende al caso de sistemas de ecuaciones, evaluando el impacto que tiene el tamaño del problema en el rendimiento del código.

De forma paralela, se documenta el uso de modelos de IA generativa como apoyo durante el desarrollo del proyecto, evaluando su utilidad real dentro de un entorno de trabajo habitual en asignaturas de programación paralela, como es el acceso remoto a servidores mediante MobaXterm.

2. Descripción del problema

El problema central consiste en calcular el gradiente de funciones polinómicas multivariables. Dado un conjunto de funciones o un sistema de funciones polinómicas de *n* variables, y un conjunto de puntos de evaluación, el objetivo es obtener para cada función y cada punto el vector de derivadas parciales:

$$\nabla f(x_1, x_2, \ldots, x_n) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n}\right)$$

2.1. Definición matemática

Para nuestro problema entendemos que la función polinómica tiene la siguiente estructura:

$$f(x_1, x_2, \ldots, x_n) = \sum_{k}^{T} c_k \prod_{i \in V_i} x_i$$

donde:

- c_k es el coeficiente del término *k*

- V_i es el conjunto de variables que intervienen en dicho término
- ***T*** es el número total de términos de la función

El gradiente se obtiene derivando cada término respecto a cada variable, aplicando la regla del producto cuando corresponda.

2.2. Restricciones y supuestos del problema

Para el desarrollo de la práctica se asumen las siguientes restricciones y supuestos:

- Las funciones son polinomios multivariables, sin términos trascendentes ni funciones no diferenciables.
- Cada función tiene un número finito de términos con variables explícitas.
- Los valores de las variables en los puntos de evaluación son reales y pueden ser cero.
- El código secuencial base solo admite términos de grado 1 o 0 en las variables. Esta decisión se toma para simplificar el cálculo y mejorar la eficiencia.
  Aunque el formato de datos permitiría extender el cálculo a términos de mayor grado mediante funciones como `pow()`, esto podría provocar un uso excesivo de memoria y un aumento significativo del tiempo de ejecución en sistemas grandes.
- Las versiones paralelas del algoritmo se basan directamente en la estructura del código secuencial, sin modificar la lógica matemática ni aplicar optimizaciones algebraicas adicionales.

3. Diseño e implementación del algoritmo

3.1. Estructura general del programa

El programa está diseñado de manera modular, hemos separado la lógica de cálculo, la lectura de datos y la ejecución de los modos en los que se espera que se pueda ejecutar el programa.
La estructura general es la siguiente:

- **Archivos de definición y tipos:**
  - `definitions.h` contiene las estructuras de datos principales (`Funcion`, `Termino`) y también
    - **`Funcion`**: representa una función polinómica compuesta por varios términos
    - **`Termino`**: representa un término individual de un polinomio, contiene un coeficiente, las variables que lo componen y el tamaño de cuantas variables debe tener
  - `type.h`: define el tipo de dato usado para los cálculos numéricos (type_data)

- **Lectura de datos:** La lectura de funciones y puntos se implementa en `fichero_common.cpp`.
  - Las funciones `lector_func()` y `lector_puntos()` permiten cargar desde ficheros situados en la carpeta `data` las funciones polinómicas y los puntos donde se evaluarán los gradientes.
  - Estas funciones realizan validaciones básicas y gestionan la reserva dinámica de memoria necesaria.
  - El formato de los fichero sería el siguiente:

La lectura de funciones leerá un fichero siguiendo este formato:

```
1 3 # num_funciones num_variables
4
3 1 1 0         # 3xy
4 0 1 1         # 4yz
2 1 0 0         # 2 *x
5 0 0 0         # 5 * x^0 * y^0
```

En la primera línea definimos el número de las funciones. La lectura de puntos leerá un fichero con este formato:

```
#v1 v2 v3
1 2 3
```

Este fichero necesita los siguientes valores: v1, v2…vn en base del `num_var` de forma directa en memoria y facilita la evaluación de secuencial como paralela

- **Ejecución de cálculos**: Cada modalidad de ejecución está encapsulada en su propio módulo:
  - `fichero_cpu` *: ejecución secuencial
  - `fichero_omp` *: ejecución paralelizada en CPU con OpenMP
  - `fichero_gpu` *: ejecución paralelizada en GPU con CUDA

- **Flujo principal:** el fichero `main.cpp` se encarga de:
  - **Leer los argumentos de entrada**
    - Esto ya venía dado por el profesor en el código base, no hemos modificado mucho al respecto
  - **Seleccionar el modo de ejecución**
    - como se puede ver en el siguiente punto que habla sobre la ejecución correspondiente, seleccionamos el modo a través de un switch siendo que si el usuario introduce
      - "-m 0" quiere decir que se ejecutará secuencialmente
      - "-m 1" indica que se ejecutará con OpenMP
      - "-m 2" indica que se ejecutará con CUDA
  - **Cargar los datos desde los ficheros**
    - Inicializamos las variables que se usarán a lo largo del programa y cargaremos los datos desde las funciones `lector_func` y `lector_puntos`

```
int num_func = 0, num_var = 0, num_puntos = 0;
Funcion *funciones = lector_func(ffuncion, &num_func, &num_var);
if(!funciones){
  printf("Error leyendo puntos desde %s\n", ffuncion);
  return -1;
}

type_data **puntos = lector_puntos(fpuntos, &num_puntos, &num_var)
if(!puntos){
  printf("Error leyendo los puntos desde %s\n", fpuntos);
  return -1;
}
```

  - **Lanzar la ejecución correspondiente**
    - Elegida por el usuario en un switch

```
switch(mode){
  case 0:
    printf("\nEJECUCION ALGORITHM SEQUENTIAL MODE...\n");
    printf("***************************************************\n");

    time_init = wtime();
    resultado = ejecucion_CPU(puntos, num_puntos, num_var, funciones, num_func);
    time_end= wtime()-time_init;

    break;
  case 1:
    printf("\nEJECUCION ALGORITHM OPEN-MP MODE...\n");
    printf("***************************************************\n");

    time_init = wtime();
    resultado = ejecucion_OMP(puntos, num_puntos, num_var, funciones, num_func, th);
    time_end= wtime()-time_init;

    break;
  case 2:
    printf("\nEJECUCION ALGORITHM GPU MODE...\n");
    printf("***************************************************\n");

    time_init = wtime();
    //insertar blockSize
    resultado = ejecucion_GPU(puntos, num_puntos, num_var, funciones, num_func, blockSize);

    time_end= wtime()-time_init;

    break;
  default:
    printf("tipo de modo de ejecucion: %d. Usa -h o -? para recibir ayuda", mode);
    return 0;
}
```

- Medir el tiempo de cálculo
  - El código mide el tiempo de ejecución en cada tipo de función permitiendo medir la escalabilidad de la misma

```
time_init = wtime();
//insertar blockSize
resultado = ejecucion_GPU(pur

time_end= wtime()-time_init;
```

  - Luego se imprimirá por pantalla el tiempo total antes de mostrar el resultado de la ejecución

```
printf("\nTiempo: %f seg\n", time_end);
```

- **Imprimir por pantalla los resultados:**

- El código indicará qué punto se ha leído desde 0 a N, para luego indicar que función desde 0 a N, esto quiere decir que si se entregan 2 puntos y 2 funciones el punto 0 recorrerá la función 0 y la función 1 del fichero dado anteriormente y luego hará lo mismo para el punto 1.

```
    printf("\nTiempo: %f seg\n", time_end);

    printf("\nGradiente calculado:\n");

   for(int p = 0; p < num_puntos; p++){
   for(int f = 0; f < num_func; f++){
   printf("Punto %d, Func %d: ", p, f);
   for(int v = 0; v < num_var; v++){
   int idx = p*num_func*num_var + f*num_var + v;
   printf("%.2f ", resultado[idx]);
   }
printf("\n");
   }
```

3.2. *Implementación secuencial*

La implementación secuencial consiste en una función en C que calcula los gradientes a través de la función `ejecucion_CPU`. Esta función evalúa un conjunto de funciones polinómicas en múltiples puntos y calcula sus derivadas parciales respecto a cada variable. La estructura general es la siguiente:

1. **Inicialización:** Se reserva memoria para almacenar todos los gradientes parciales:

```
type_data *grad = (type_data*)calloc(num_puntos * num_func * num_var,
sizeof(type_data));
if (grad == NULL) return NULL;
```

2. **Recorrido de puntos**: Se itera sobre cada punto de evaluación haciendo que cada punto se evalúe de forma independiente
3. **Recorrido de funciones**: Para cada función se recorren todos los términos que tiene cada función. Por cada punto se recorren todas las funciones del sistema
4. **Recorrido de los términos de una función**
    - Cada término contiene:
        i. coeficiente
        ii. lista de variables que intervienen en el producto

```
//Recorrer los puntos
for(int p=0;p<num_puntos;p++) {

  //recorrer las funciones para luego recorrer las variables para calcular la derivada parcial
  for(int f=0;f<num_func;f++) {

    //Recorrer los terminos de una funcion
    for(int t=0; t<funciones[f].max_terms;t++){
      Termino *term = &funciones[f].terms[t];
```

5. **Cálculo de derivadas parciales**
   - Para cada variable del término, se calcula su derivada usando la regla de la derivada del producto

```
    //Derivamos con respecto a cada variable que existe en el termino
     for(int i=0;i<term->tam_var;i++){
      int v_derivada  = term->var[i];
      type_data valor_derivada = term->coef;

    //Multiplicamos por las demas variables del mismo termino
    for(int j=0;j<term->tam_var;j++){
      if( i != j){
  int v_actual = term->var[j];
  valor_derivada *= puntos[p][v_actual];
}
```

   - Las derivadas resultantes se acumulan en grad.

```
grad[p*num_func*num_var+f*num_var+v_derivada] += valor_derivada;
```

3.3. Paralelización en CPU con OpenMP

La implementación de la paralelización con OpenMP ocurre en `ejecucion_OMP`, encargada de calcular los gradientes parciales de un conjunto de funciones polinómicas evaluadas en varios puntos, utilizando OpenMP para aprovechar la computación en paralelo en CPU.El número de hilos a utilizar se define en la línea:

```
omp_set_num_threads(th)
```

donde `th` es el número de hilos solicitado por el usuario. Esta instrucción permite controlar explícitamente la cantidad de hilos que ejecutarán la región paralela.

3.3.1. Paralelización principal

Se paraleliza el bucle externo que recorre los puntos::

```
#pragma omp parallel for schedule (static)
```

El uso de schedule(static) permite repartir los puntos de forma equitativa entre los hilos. Cada hilo calcula los gradientes de los puntos que le han sido asignados, evitando conflictos de escritura en memoria

```
//valores a calcular el gradiente //num de variables //funcion(es) //num de funciones //numero de puntos a los que hacer gradiente
type_data *ejecucion_OMP(type_data **puntos, int num_puntos,  int num_var, Funcion *funciones, int num_func, int th){

  omp_set_num_threads(th);

  type_data *grad = (type_data*)calloc(num_puntos * num_func * num_var, sizeof(type_data));
  if(grad == NULL) return NULL;

  //PARALELIZAR PUNTOS PARA QUE CADA HILO PROCESE PUNTOS DISTINTOS
  #pragma omp parallel for schedule(static)
  //Recorrer los puntos
  for(int p=0;p<num_puntos;p++) {

    //recorrer las funciones para luego recorrer las variables para calcular la derivada parcial
    for(int f=0;f<num_func;f++) {

      //Recorrer los terminos de una funcion
      for(int t=0; t<funciones[f].max_terms;t++){
        Termino *term = &funciones[f].terms[t];

        //Producto total del termino en el punto P
        type_data total = term->coef;
        for(int i=0;i<term->tam_var;i++){
          int v = term->var[i];
          total *= puntos[p][v];
        }

        //Derivadas parciales
        for(int i=0;i<term->tam_var;i++){
          int v = term->var[i];


          if(v < 0 || v >= num_var || puntos[p][v] == 0.0) continue;

          type_data derivada = total/puntos[p][v];

          grad[p*num_func*num_var+f*num_var+v] += derivada;
        }
      }
    }
  }
  return grad;
}
```

3.3.2. Ventajas y limitaciones

Ventajas:

- Paralelización sencilla y efectiva para un número grande de puntos.
- No requiere sincronización compleja al solo paralelizar el código en base a los puntos.
- Cada hilo se escribe en una región independiente de memoria.

Limitaciones

- Si `num_puntos` es pequeño, algunos hilos pueden quedarse inactivos, limitando la escalabilidad
- schedule(static) puede generar desequilibrio si algunos puntos tardan más en evaluarse
- La paralelización se realiza solo en los puntos, pero si `num_puntos` es menor a `num_hilos`, la CPU no se aprovecha completamente
- No contempla términos polinómicos con exponentes distintos de 1 en las variables.

3.4. Paralelización en GPU con CUDA

Las estructuras originales `Termino` y `Funcion` contienen punteros internos, lo que impide su uso directo en la GPU.Para solucionarlo hemos definido versiones adaptadas, que sean “GPU-friendly” (`Termino_GPU` y `Funcion_GPU`), basadas en arrays planos contiguos, lo que garantiza accesos coalescentes y evita errores del puntero inválidos en device code.

```
//Estructuras GPU:
typedef struct {
    type_data coef;
    int *var_idx;
    int num_var;
} Termino_GPU;

typedef struct {
  int start_term;
  int max_terms;
} Funcion_GPU;
```

Para simplificar la implementación se usó memoria unificada mediante `cudaMallocManaged`, debido a que permite asignar memoria unificada accesible tanto desde CPU como GPU. Esta solución simplifica la implementación, aunque puede penalizar el rendimiento frente a una gestión manual de memoria en casos de gran volumen de datos. Las regiones reservadas fueron las siguientes:

- **puntos**: matriz de puntos a evaluar, aplanado en un vector 1D.
- **funcion_GPU**: array de funciones adaptadas
- **termino_GPU**: array contiguo con todos los términos
- **var_GPU**: vector con todas las variables de todos los términos
- grad: vector de salida con todos los gradientes

Para el esquema de paralelización se hizo un enfoque thread-per-element, es decir, cada hilo calcula exactamente una derivada parcial en base a:

- un punto `p`
- una función `f`
- una variable `v`

El índice global del hilo se calcula como:

$$idx = p * (num_{func} * num_{var}) + f * num_{var} + v$$

```
__global__ void kernel_gradiente(const type_data *puntos, const Funcion_GPU *funciones, const Termino_GPU *terminos, type_data *grad, int num_puntos, int num_var, int num_func)
{
  int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // Total de hilos = num_puntos * num_func * max_terms
  int total = num_puntos * num_func * funciones[0].max_terms;
  if (idx >= total) return;

    // Descomponer índice lineal
  int p = idx / (num_func * funciones[0].max_terms);
  int rest = idx % (num_func * funciones[0].max_terms);
  int f = rest / funciones[0].max_terms;
  int t = rest % funciones[0].max_terms;

  // Obtener el término
  Termino_GPU term = terminos[ funciones[f].start_term + t ];

  // Producto total del término
  type_data total_prod = term.coef;
  for (int i = 0; i < term.num_var; i++) {
    int v = term.var_idx[i];
    total_prod *= puntos[p * num_var + v];
  }

  // Derivadas parciales
  for (int i = 0; i < term.num_var; i++) {
    int v = term.var_idx[i];
    type_data x = puntos[p * num_var + v];

    if (x == 0.0) continue;

    type_data deriv = total_prod / x;

    // Acceso concurrente con atomicAdd
    atomicAdd(&grad[p * num_func * num_var + f * num_var + v], deriv);
  }
}
```

El kernel recorre los términos de la función correspondiente y acumula la derivada parcial. En la configuración del kernel el número de hilos por bloques se recibe como parámetro, haciendo que el número de bloques total se calcula tal que:

$$gridSize = (total_threads + blockSize - 1) / blockSize$$

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;

  // Total de hilos = num_puntos * num_func * max_terms
int total = num_puntos * num_func * funciones[0].max_terms;
if (idx >= total) return;


  // Descomponer índice lineal
int p = idx / (num_func * funciones[0].max_terms);
int rest = idx % (num_func * funciones[0].max_terms);
int f = rest / funciones[0].max_terms;
int t = rest % funciones[0].max_terms;

// Obtener el término
Termino_GPU term = terminos[ funciones[f].start_term + t ];

// Producto total del término
type_data total_prod = term.coef;
for (int i = 0; i < term.num_var; i++) {
  int v = term.var_idx[i];
  total_prod *= puntos[p * num_var + v];
}

// Derivadas parciales
for (int i = 0; i < term.num_var; i++) {
  int v = term.var_idx[i];
  type_data x = puntos[p * num_var + v];

  if (x == 0.0) continue;

  type_data deriv = total_prod / x;

  // Acceso concurrente con atomicAdd
  atomicAdd(&grad[p * num_func * num_var + f * num_var + v], deriv);
}
```

Ahora comenzaría la ejecución de la función ejecución_CPU. Para comenzar se reservan memoria para los gradientes, luego se copian los puntos en un array plano, para finalmente construir las estructuras de la GPU.

```
// Copiar puntos a array plano
type_data *puntos_d;
cudaMalloc(&puntos_d, num_puntos * num_var * sizeof(type_data));

type_data *puntos_flat = (type_data*)malloc(num_puntos * num_var * sizeof(type_data));
for (int p = 0; p < num_puntos; p++)
  for (int v = 0; v < num_var; v++)
    puntos_flat[p * num_var + v] = puntos_h[p][v];

cudaMemcpy(puntos_d, puntos_flat, num_puntos * num_var * sizeof(type_data), cudaMemcpyHostToDevice);

free(puntos_flat);

// Construir estructuras GPU
int total_terms = 0;
for (int f = 0; f < num_func; f++)
  total_terms += funciones_h[f].max_terms;

Funcion_GPU *funciones_d;
Termino_GPU *terminos_d;

cudaMalloc(&funciones_d, num_func * sizeof(Funcion_GPU));
cudaMalloc(&terminos_d, total_terms * sizeof(Termino_GPU));

// Copiar funciones y términos
Funcion_GPU *funciones_tmp = (Funcion_GPU*)malloc(num_func * sizeof(Funcion_GPU));
Termino_GPU *terminos_tmp = (Termino_GPU*)malloc(total_terms * sizeof(Termino_GPU));

int term_offset = 0;
for (int f = 0; f < num_func; f++) {
  funciones_tmp[f].start_term = term_offset;
  funciones_tmp[f].max_terms = funciones_h[f].max_terms;

  for (int t = 0; t < funciones_h[f].max_terms; t++) {
    Termino *term = &funciones_h[f].terms[t];
    terminos_tmp[term_offset].coef = term->coef;
    terminos_tmp[term_offset].num_var = term->tam_var;

    // Copiar índices de variables
    int *var_idx_d;
    cudaMalloc(&var_idx_d, term->tam_var * sizeof(int));
    cudaMemcpy(var_idx_d, term->var, term->tam_var * sizeof(int), cudaMemcpyHostToDevice);

    terminos_tmp[term_offset].var_idx = var_idx_d;
    term_offset++;
    }
}
```

Una vez construidas las estructuras de la GPU podemos lanzar el kernel:

```
// Lanzar kernel
int total_threads = num_puntos * num_func * funciones_h[0].max_terms;
int gridSize = (total_threads + blockSize - 1) / blockSize;

kernel_gradiente<<<gridSize, blockSize>>>(puntos_d, funciones_d, terminos_d, grad, num_puntos, num_var, num_func);

cudaDeviceSynchronize();

// Copiar resultado al host
type_data *grad_h = (type_data*)malloc(total_grad * sizeof(type_data));
cudaMemcpy(grad_h, grad, total_grad * sizeof(type_data), cudaMemcpyDeviceToHost);

// Liberar GPU
cudaFree(puntos_d);
cudaFree(funciones_d);
cudaFree(terminos_d);
cudaFree(grad);

return grad_h;
```

Tras el lanzamiento se debe de copiar el resultado en el host y ya podremos liberar la GPU

4. Resultados Experimentales

4.1. Entorno de ejecución

Los experimentos se realizaron en el servidor luna de la UCAM usando MobaXterm para el acceso remoto. Las características del entorno fueron las siguientes:

1. **Hardware**
   - **CPU**: 2 sockets x 6 núcleos cada uno, Intel Xeon E5-2603 v4, 1.7GHz y 12 hilos totales
   - **RAM**: 31 GB
   - **GPU**: NVIDIA GeForce RTX 2080 Ti, 11GB RAM disponible
   - **Arquitectura**: x86_64, 64 bits

2. **Sistema Operativo**
   - Ubuntu 22.04.5 LTS (Jammy Jellyfish), kernel 5.15.0-161-generic.

3. **Compilación y herramientas**
   - **Compiladores:** `g++` para código CPU y `nvcc` (CUDA Toolkit 12.2) para código GPU.
   - **Flags de compilación CPU:** `-O0 -m64 -fopenmp` para la versión paralela con OpenMP.
   - **Flags de compilación CUDA:** `-O0 -m64 -gencode arch=compute_75,code=sm_75 --cudart=shared.`
   - **Includes y librerías:** `-I/usr/local/cuda/include, -lcudart, -lcublas, -lcurand, -lstdc++.`

4. **Comandos de ejecución (ejemplos):**
   - `./fichero_ejecutable -s puntos10k -o funcion1k -m 0`
   - `./fichero_ejecutable -s puntos10k -o funcion1k -m 1 -t 1`
   - `./fichero_ejecutable -s puntos10k -o funcion1k -m 2 -b 256`
5. **Salida**

```
Punto …, Func …: ….00, ….00, ….00
```

4.2. Resultados: Validación del cálculo del gradiente

Para comprobar que el cálculo del gradiente es correcto

1. **Primer Ejemplo de función polinómica simple:**

$$f(x,y) = 4xy + 3y$$

Si calculamos el gradiente para el punto (2,3):

$$\frac{df}{dx} = 4y \, , \frac{df}{dy} = 4x + 3$$

El resultado esperado debería ser: **(12, 11)**

El fichero con la función se llamará: `funcion_simple1` cuyo contenido será:

```
1 2
2
4 1 1
3 0 1
```

Al ejecutar el código para el punto que hemos dicho antes (`punto_simple1`) podemos ver que el resultado es **correcto** en los tres casos

```
g26@luna:~/recuperacion/practicasPP-25-26$
g26@luna:~/recuperacion/practicasPP-25-26$ ./fichero_ejecutable  -o funcion_simple1 -s punto_simple1 -m 0
DATA. rows 1 columns 2


EJECUCION ALGORITHM SEQUENTIAL MODE...
************************************************

Tiempo: 0.000001 seg

Gradiante calculado:
Punto 0, Func 0: 12.00 11.00
g26@luna:~/recuperacion/practicasPP-25-26$ ./fichero_ejecutable  -o funcion_simple1 -s punto_simple1 -m 1 -t 128
DATA. rows 1 columns 2


EJECUCION ALGORITHM OPEN-MP MODE...
************************************************

Tiempo: 0.008747 seg

Gradiante calculado:
Punto 0, Func 0: 12.00 11.00
g26@luna:~/recuperacion/practicasPP-25-26$ ./fichero_ejecutable  -o funcion_simple1 -s punto_simple1 -m 2 -b 256
DATA. rows 1 columns 2


EJECUCION ALGORITHM GPU MODE...
************************************************

Tiempo: 0.271333 seg

Gradiante calculado:
Punto 0, Func 0: 12.00 11.00
g26@luna:~/recuperacion/practicasPP-25-26$
```

2. **Segundo Ejemplo de función polinómica simple**

$$f(x, y, z) = 2xyz + 5x + 4z$$

Si calculamos el gradiente para el punto (1, 2, 3):

$$\frac{df}{dx} = 2yz + 5 \, , \frac{df}{dy} = 2xz \, , \frac{df}{dz} = 2xy + 4$$

El resultado esperado debería ser: **(17, 6, 8)**

El fichero con la función se llamará: `funcion_simple2` cuyo contenido será:

```
1 3
3
2 1 1 1
5 1 0 0
4 0 0 1
```

Al ejecutar el código para el punto que hemos dicho antes (`punto_simple2`) podemos ver que el resultado es **correcto** en los tres casos

```
g26@luna:~/recuperacion/practicasPP-25-26$ ./fichero_ejecutable  -o funcion_simple2 -s punto_simple2 -m 0
DATA. rows 1 columns 3


EJECUCION ALGORITHM SEQUENTIAL MODE...
************************************************

Tiempo: 0.000002 seg

Gradiante calculado:
Punto 0, Func 0: 17.00 6.00 8.00
g26@luna:~/recuperacion/practicasPP-25-26$ ./fichero_ejecutable  -o funcion_simple2 -s punto_simple2 -m 1 -t 8
DATA. rows 1 columns 3


EJECUCION ALGORITHM OPEN-MP MODE...
************************************************

Tiempo: 0.000459 seg

Gradiante calculado:
Punto 0, Func 0: 17.00 6.00 8.00
g26@luna:~/recuperacion/practicasPP-25-26$ ./fichero_ejecutable  -o funcion_simple2 -s punto_simple2 -m 2 -b 256
DATA. rows 1 columns 3


EJECUCION ALGORITHM GPU MODE...
************************************************

Tiempo: 0.273337 seg

Gradiante calculado:
Punto 0, Func 0: 17.00 6.00 8.00
```

3. Primer Ejemplo de Sistema de funciones

f_1: $3xy + 2x$

f_2: $x + y$

Si calculamos el gradiente para el punto (1, 2):

f_1: $\frac{df}{dx} = 3y + 2$, $\frac{df}{dy} = 3$

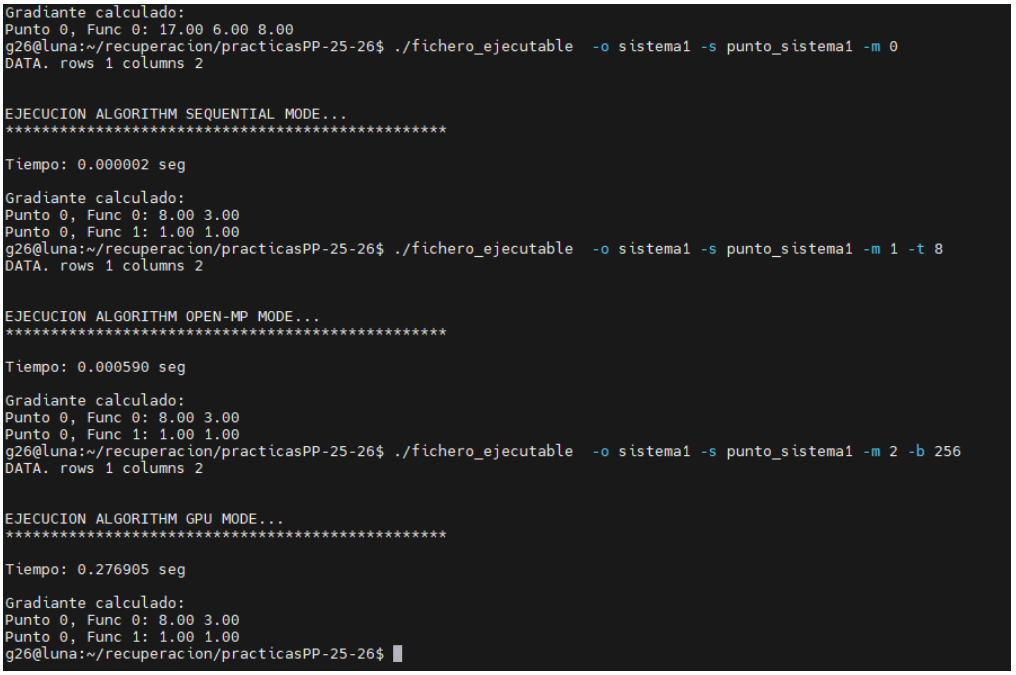
f_2: $\frac{df}{dx} = 1$, $\frac{df}{dy} = 1$

El resultado esperado debería ser: **(8, 3) y (1, 1)**

El fichero con la función se llamará: `sistema1` cuyo contenido será:

```
2 2
2
3 1 1
2 1 0
2
1 1 0
1 0 1
```

Al ejecutar el código para el punto que hemos dicho antes (`punto_sistema1`) podemos ver que el resultado es **correcto** en los tres casos



4. Segundo Ejemplo de Sistema de funciones

$f_1: xy + z$

$f_2: 2xz$

$f_1: 3yz + x$

Si calculamos el gradiente para el punto (1, 2, 3)

$f_1: \frac{df}{dx} = y\ , \frac{df}{dy} = x, \frac{df}{dz} = 1$

$f_2: \frac{df}{dx} = 2z\ , \frac{df}{dy} = 0, \frac{df}{dz} = 2$

$f_3: \frac{df}{dx} = 1\ , \frac{df}{dy} = 3z, \frac{df}{dz} = 3y$

El resultado esperado debería ser: **(2, 1,1), (6, 0, 2), (1, 9, 6)**

El fichero con la función se llamará: `sistema2` cuyo contenido será:

```
3 3
2
1 1 1 0
1 0 0 1
2
2 1 0 1
0 0 0 0
2
3 0 1 1
1 1 0 0
```

Al ejecutar el código para el punto que hemos dicho antes (`punto_sistema2`) podemos ver que el resultado es **correcto** en los tres casos

```
g26@luna:~/recuperacion/practicasPP-25-26$ ./fichero_ejecutable  -o sistema2 -s punto_sistema2 -m 0
DATA. rows 1 columns 3


EJECUCION ALGORITHM SEQUENTIAL MODE...
***********************************************

Tiempo: 0.000002 seg

Gradiante calculado:
Punto 0, Func 0: 2.00 1.00 1.00
Punto 0, Func 1: 6.00 0.00 2.00
Punto 0, Func 2: 1.00 9.00 6.00
g26@luna:~/recuperacion/practicasPP-25-26$ ./fichero_ejecutable  -o sistema2 -s punto_sistema2 -m 1 -t 8
DATA. rows 1 columns 3


EJECUCION ALGORITHM OPEN-MP MODE...
***********************************************

Tiempo: 0.000568 seg

Gradiante calculado:
Punto 0, Func 0: 2.00 1.00 1.00
Punto 0, Func 1: 6.00 0.00 2.00
Punto 0, Func 2: 1.00 9.00 6.00
g26@luna:~/recuperacion/practicasPP-25-26$ ./fichero_ejecutable  -o sistema2 -s punto_sistema2 -m 2 -b 256
DATA. rows 1 columns 3


EJECUCION ALGORITHM GPU MODE...
***********************************************

Tiempo: 0.266710 seg

Gradiante calculado:
Punto 0, Func 0: 2.00 1.00 1.00
Punto 0, Func 1: 6.00 0.00 2.00
Punto 0, Func 2: 1.00 9.00 6.00
g26@luna:~/recuperacion/practicasPP-25-26$
```

5. Tercer Ejemplo: Gradientes con varios puntos

Para este ejemplo he decidido añadir otro punto más para comprobar que calcula correctamente una función (`funcionA`) por diferentes puntos:

$$f_{(x,y,z)} = 3xy + 4yz + 2x + 5$$

```
1 3
4
3 1 1 0
4 0 1 1
2 1 0 0
5 0 0 0
```

La gradiente general sería

$$\frac{df}{dx} = 3y + 2 \, , \frac{df}{dy} = 3x + 4z \, , \frac{df}{dz} = 4y$$

- El gradiente para el punto (1,2,3) es: (8.15.8)
- El gradiente para el punto (2, 1, 4) es (5, 22, 4)

```
1 2 3
2 1 4
```

Al ejecutar el código en sus tres modos podemos ver que los resultados son totalmente correctos

```
g26@luna:~/recuperacion/practicasPP-25-26$ ./fichero_ejecutable  -o funcionA -s puntosA -m 0
DATA. rows 2 columns 3


EJECUCION ALGORITHM SEQUENTIAL MODE...
***********************************************

Tiempo: 0.000002 seg

Gradiante calculado:
Punto 0, Func 0: 8.00 15.00 8.00
Punto 1, Func 0: 5.00 22.00 4.00
g26@luna:~/recuperacion/practicasPP-25-26$ ./fichero_ejecutable  -o funcionA -s puntosA -m 1 -t 8
DATA. rows 2 columns 3


EJECUCION ALGORITHM OPEN-MP MODE...
***********************************************

Tiempo: 0.000548 seg

Gradiante calculado:
Punto 0, Func 0: 8.00 15.00 8.00
Punto 1, Func 0: 5.00 22.00 4.00
g26@luna:~/recuperacion/practicasPP-25-26$ ./fichero_ejecutable  -o funcionA -s puntosA -m 2 -b 256
DATA. rows 2 columns 3


EJECUCION ALGORITHM GPU MODE...
***********************************************

Tiempo: 0.267668 seg

Gradiante calculado:
Punto 0, Func 0: 8.00 15.00 8.00
Punto 1, Func 0: 5.00 22.00 4.00
```

4.3. Resultados: rendimiento y escalabilidad

Se presentan los tiempos de ejecución obtenidos para los distintos modos del programa: secuencial, paralelización con OpenMP y paralelización con CUDA.

Resultados del modo secuencial:

El modo secuencial se ejecutó utilizando distintos tamaños de entrada, combinando diferentes cantidades de puntos y funciones. Los tiempos obtenidos son los siguientes:

:

Puntos × Funciones	Tiempo (ms)
100 × 10	0.191
100 × 100	1,935
100 × 1000	18,742
1000 × 10	2,571
1000 × 100	23,292
1000 × 1000	189,182
10000 × 10	22,532
10000 × 100	186,478
10000 × 1000	1.824,377

El código secuencial trabaja correctamente con ficheros de hasta diez mil puntos y sistemas de hasta mil funciones, registrando un incremento progresivo del tiempo de ejecución conforme aumenta el tamaño del problema.

Modo Paralelizado con OpenMP:

Para evaluar el comportamiento del código paralelo con OpenMP, se usó el caso más grande (puntos10k y funcion1k) y se varió el número de hilos empleados en la ejecución.

Hilos	Tiempo (ms)
1	1805,806
4	520,580
8	289,985
16	279,250
32	245,327
64	237,312
128	242,574
256	242,035
512	259,358

1024	275,596
2048	329,214
4096	426,351

Modo Paralelización CUDA:

Se realizaron varias pruebas variando el tamaño del bloque en la ejecución CUDA y se cambió el tamaño de los puntos, de diez mil a cincuenta mil para apreciar la optimización del código

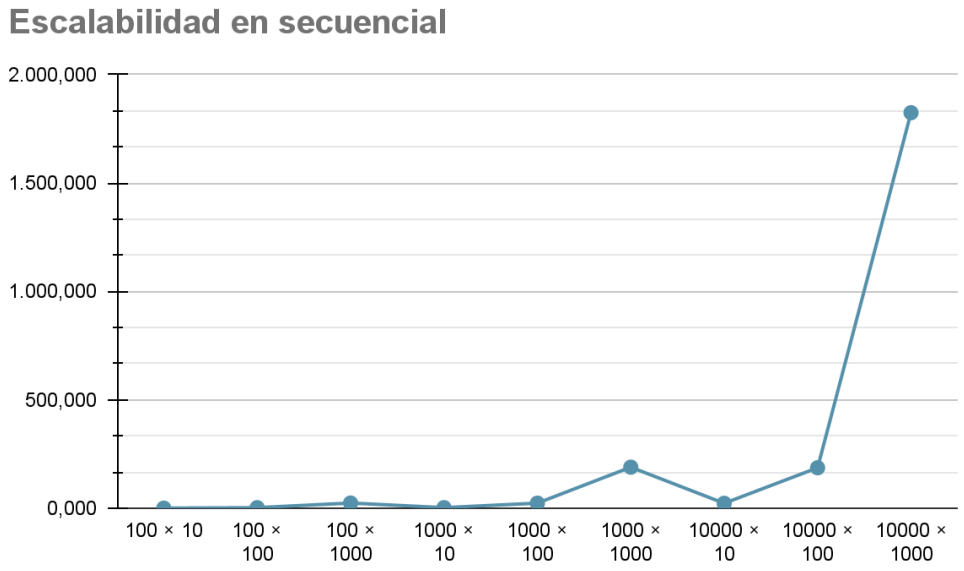
Bloque	Tiempo (s) 50k puntos	Tiempo (s) 10k puntos
1	1,10254	0,463892
2	1,046938	0,45249
4	0,966205	0,439194
8	0,927925	0,432677
16	0,913344	0,417424
32	0,901455	0,41441
64	0,90333	0,415184

128	0,891514	0,417078
256	0,902138	0,423203
512	0,902524	0,418575
1024	0,903282	0,424441

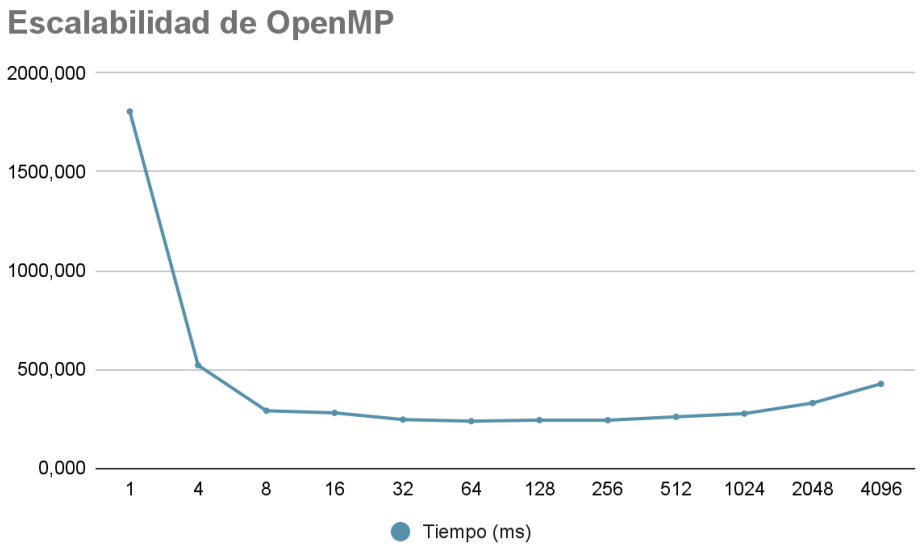
El número de puntos se aumentó debido a que no había una mejora apreciable entre los diferentes modos. Debido a ellos probamos a aumentar el número de puntos de diez mil a cincuenta mil.

4.4. Análisis de los resultados

En el **código secuencial (CPU)**, el tiempo de ejecución crece de forma clara conforme aumenta el tamaño del problema debido a que el cálculo del gradiente se realiza de manera completamente serial. Al no aprovechar el hardware multicure, la escalabilidad de esta versión queda muy limitada al tamaño de los ficheros de entrada.



En el caso del **código paralelo con OpenMP**, se observa una reducción muy significativa en el tiempo de ejecución al incrementar el número de hilos. La mejora es especialmente notable en datasets grandes, donde el trabajo por hilo es suficiente para compensar el coste de la paralelización. A partir de cierto número de hilos, el tiempo comienza a estabilizarse e incluso a aumentar ligeramente, debido al overhead de gestión de hilos, la reducción del trabajo útil por hilo y la mayor presión sobre la jerarquía de memoria y las cachés.



El mejor rendimiento se obtiene cuando el número de hilos supera ligeramente el número de núcleos físicos disponibles, antes de que el overhead asociado a la paralelización domine el tiempo total de ejecución.

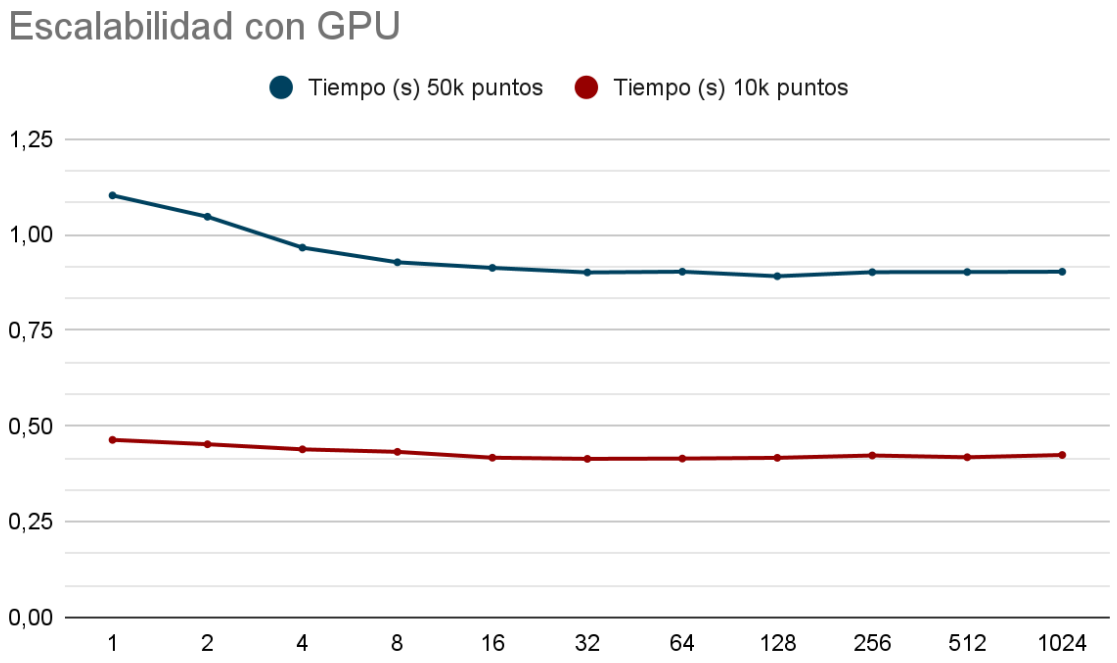
En la **versión CUDA**: El gráfico demuestra que el tiempo de ejecución se mantiene estable, el comportamiento demuestra que si bien, tarda bastante más que con otros modos de ejecución (se mantiene en los 0.9 segundos de ejecución) si lo comparamos con otros modos de ejecución el código tardará mucho más en ser ejecutado.

- **Ejecución en OpenMP:** Se ejecutaron pruebas de ejecución, los resultados demuestran que el tiempo de ejecución es mayor al de la GPU:

Hilos	Tiempo (s)
1	9,004469
2	4,813820
4	2,553873
16	1,172826
64	1,062117
128	1,060307
256	1,071785
1024	1,115683

- **Ejecución en CPU:** 9120,242 ms

Hay una mejora, que claramente es notoria cuando estamos trabajando con un mayor volumen de datos, superando los tiempos de ejecución de OpenMP y de CPU.



5. Conclusiones

La paralelización mediante OMP ha demostrado ser una solución muy efectiva frente a la CPU, consiguiendo reducciones de tiempo muy significativas al aumentar el número de hilos. Se puede observar en los experimentos que la mejora es clara, hasta cierto punto, ya que el rendimiento tiende a estabilizarse o incluso empeorar ligeramente debido al overhead de gestión de hilos. Aún así, OpenMP demuestra ser una opción muy adecuada para problemas de tamaño pequeño y medio en sistemas multiprocesador.

Por otro lado, la implementación en GPU con CUDA demuestra mayor potencial para la aceleración cuando el volumen de datos es suficientemente alto. En los experimentos con cincuenta mil puntos, la GPU consigue tiempos de en torno a 0,9 segundos, superando claramente a la versión secuencial y a la paralelizada con OpenMP. Confirmando que el enfoque thread-per-element que hemos usado se adapta bien a la arquitectura masivamente paralela de la GPU, permitiendo explorar miles de hilos concurrentes para calcular derivadas parciales de forma independiente.Sin embargo, se ha observado que en tamaños de

entrada medios o pequeños el uso de CUDA no resulta ventajoso muy posiblemente debido al coste de inicialización, transferencia y gestión de memoria, lo que provoca que el tiempo total sea comparable o incluso peor que el de OpenMP. Esto significa que la GPU solo compensa cuando el problema es bastante grande como para amortizar dichos costes. Finalmente, el uso de memoria unificada ha simplificado considerablemente la implementación de CUDA, aunque posiblemente limita el rendimiento máximo alcanzable.

En conjunto, los resultados demuestran que la paralelización es clave para el cálculo eficiente de gradientes en sistemas grandes, siendo OpenMP ideal para acelerar el código en CPU de forma sencilla y robusta mientras que CUDA es la mejor opción para cuando se trabaja con volúmenes masivos de datos y se dispone de una GPU potente-